FEMaster User Manual

Finite Elements in Mechanical Analysis and Simulation for Technical Engineering Research

Working Draft

28 April 2026

Contents

1 Overview **1**

1.1 Model layers . . . 1

1.2 Supported analysis types . . . 1

1.3 Degree-of-freedom convention . . . 2

1.4 Targets . . . 2

2 Build and Command Line **3**

2.1 Build . . . 3

2.2 Running an input deck . . . 3

2.3 DSL documentation mode . . . 3

3 DSL Basics **5**

3.1 Commands . . . 5

3.2 Scopes . . . 5

3.3 Aliases, defaults, and flags . . . 5

3.4 Component order . . . 5

3.5 Targets . . . 6

4 Command Reference **7**

4.1 Geometry and topology . . . 7

4.2 Properties . . . 7

4.3 Features, fields, loads, and constraints . . . 7

4.4 Loadcase controls . . . 8

5 Nodes **9**

5.1 Node IDs . . . 9

5.2 Coordinate meaning . . . 9

5.3 Active degrees of freedom . . . 9

6 Elements **11**

6.1 Registered element types . . . 11

6.2 Solid element family . . . 11

6.3 `C3D4`: 4-node tetrahedral solid . . . 12

6.4 `C3D5`: 5-node pyramid input . . . 12
6.5 `C3D6`: 6-node wedge solid . . . 13
6.6 `C3D8`: 8-node hexahedral solid . . . 13
6.7 `C3D10`: 10-node quadratic tetrahedral solid . . . 13
6.8 `C3D15`: 15-node quadratic wedge solid . . . 14
6.9 `C3D20`: 20-node quadratic hexahedral solid . . . 14
6.10 `C3D20R`: 20-node quadratic hexahedral solid with reduced integration . . . 15
6.11 Shell element family . . . 15
6.12 `S3`: 3-node triangular shell . . . 15
6.13 `S4`: 4-node quadrilateral shell . . . 16
6.14 `MITC4`: 4-node quadrilateral shell with tied shear . . . 16
6.15 `S6`: 6-node quadratic triangular shell . . . 16
6.16 `S8`: 8-node quadratic quadrilateral shell . . . 17
6.17 `QSPT`: quadrilateral shear-panel type . . . 17
6.18 Beam and truss element family . . . 17
6.19 `B33`: 2-node 3D beam . . . 17
6.20 `T3`: 2-node truss . . . 18
6.21 Choosing an element . . . 18

7 Surfaces **19**
7.1 Surface from one element side . . . 19
7.2 Surface from an element set . . . 19
7.3 Orientation and sign . . . 19

8 Sets **20**
8.1 Node sets . . . 20
8.2 Element sets . . . 20
8.3 Surface sets . . . 20
8.4 Naming . . . 20

9 Sections and Profiles **21**
9.1 Solid section . . . 21
9.2 Shell section . . . 21
9.3 Beam profiles and beam sections . . . 21
9.4 Truss section . . . 22

10 Features **23**
10.1 Point Mass . . . 23

11 Material Models **24**
11.1 Material context: `*MATERIAL` . . . 24
11.2 Density: `*DENSITY` . . . 24

11.3 Thermal expansion: `*THERMALEXPANSION` . . . 25
11.4 Isotropic elasticity . . . 25
11.5 Generalised isotropic elasticity . . . 25
11.6 Orthotropic elasticity . . . 26
11.7 ABD elasticity . . . 26

12 Coordinate Systems and Orientations **28**
12.1 Rectangular systems . . . 28
12.2 Cylindrical systems . . . 29
12.3 Where orientations are used . . . 29

13 Fields **30**
13.1 Typical references . . . 30

14 Loads, Supports, and Amplitudes **31**
14.1 Support collectors . . . 31
14.2 Concentrated loads . . . 31
14.3 Surface traction and pressure . . . 32
14.4 Volume loads . . . 32
14.5 Thermal loads . . . 33
14.6 Inertial loads . . . 33
14.7 Amplitudes . . . 33

15 Constraints **34**
15.1 Rigid-body motion suppression: `*RBM` . . . 34
15.2 Coupling: `*COUPLING` . . . 35
15.2.1 Kinematic coupling equations . . . 35
15.2.2 Structural coupling load distribution . . . 36
15.3 Tie: `*TIE` . . . 36
15.4 Connector: `*CONNECTOR` . . . 36
15.5 Constraint summary . . . 37

16 Analysis and Solver Controls **38**
16.1 Activating support and load collectors . . . 38
16.2 Solver . . . 38
16.3 Constraint method . . . 39
16.3.1 Nullspace method . . . 39
16.3.2 Lagrange method . . . 39
16.4 Inertia relief . . . 40
16.5 Load rebalancing . . . 40
16.6 Matrix and diagnostic requests . . . 41
16.7 Eigenvalue parameters . . . 41

16.8 Transient-specific controls . . . 42

17 Loadcases **44**

17.1 General syntax . . . 44
17.2 Common equation structure . . . 44
17.3 Linear Static . . . 44
17.3.1 Governing equation . . . 44
17.3.2 Typical DSL . . . 45
17.4 Linear Static Topology . . . 45
17.4.1 Governing equation . . . 45
17.4.2 Typical DSL . . . 45
17.5 Eigenfrequency . . . 46
17.5.1 Governing equation . . . 46
17.5.2 Typical DSL . . . 46
17.6 Linear Buckling . . . 46
17.6.1 Governing equations . . . 46
17.6.2 Sigma . . . 47
17.6.3 Typical DSL . . . 47
17.7 Linear Transient . . . 47
17.7.1 Governing equation . . . 47
17.7.2 Typical DSL . . . 48
17.8 Typical command overview . . . 48

18 Result Format **49**

18.1 Loadcase blocks . . . 49
18.2 Dense fields . . . 49
18.3 Indexed fields . . . 49
18.4 Common field names . . . 49

19 Examples **51**

19.1 Linear static . . . 51
19.2 Eigenfrequency . . . 51
19.3 Linear buckling . . . 51
19.4 Transient load with amplitude . . . 52
19.5 Point mass . . . 52

A Mathematical Derivations **53**

A.1 Element stiffness . . . 53
A.2 Compliance . . . 53
A.3 Density penalization . . . 54
A.4 Orientation-dependent stiffness . . . 54
A.5 Voigt transformation . . . 54

A.6 Orientation sensitivity 55
A.7 Rotation derivatives 55

1 Overview

FEMaster is a finite-element solver for linear mechanical analyses. Models are described with an input-deck language in which commands start with an asterisk, command options are written as keywords, and numerical data follows on subsequent lines.

The manual is organized as a user reference. Command tables summarize the available DSL commands; the later chapters explain how model entities are connected, how collectors are activated in loadcases, and how analysis controls affect the solved equations.

1.1 Model layers

A model starts with nodes. Nodes define global coordinates and provide the reference points for all other model entities. Elements connect nodes into line, surface, or volume discretizations. Surfaces describe element sides used by pressure loads, distributed loads, ties, and surface couplings. Sets group nodes, elements, or surfaces so that loads, supports, sections, and constraints can refer to meaningful names instead of long ID lists.

Materials and sections give elements their physical meaning. An element by itself defines only topology and interpolation. A section assigns material, thickness, area, or profile data. Features such as point masses are additional model objects that contribute mass, inertia, or spring stiffness without being part of element connectivity.

Loads and supports are stored in named collectors. A loadcase decides which collectors are active. This allows one model to contain several analyses without repeating geometry, materials, sections, or load definitions.

1.2 Supported analysis types

Type	Purpose
`LINEARSTATIC`	Small-displacement static equilibrium with linear material behaviour.
`LINEARSTATICTOPO`	Static analysis with element-wise topology fields such as density, orientation, and penalization exponent.
`EIGENFREQ`	Natural-frequency analysis of the linear constrained system.
`LINEARBUCKLING`	Linear buckling analysis from a static preload and the resulting geometric stiffness.
`LINEARTRANSIENT`	Linear time integration with Newmark parameters, optional Rayleigh damping, and controlled result cadence.

1.3 Degree-of-freedom convention

Mechanical nodal vectors use the component order

`Ux, Uy, Uz, Rx, Ry, Rz`.

The first three components are translations and the last three are rotations. Not every element uses all six components. Solid elements usually contribute only translational displacement degrees of freedom. Beams, shells, constraints, connectors, and some features may introduce rotations.

1.4 Targets

Many commands accept a target that can be either a set name or a single numeric ID. For example, `*CLOAD` can target a node set or one node, `*DLOAD` can target a surface set or one surface, and `*VLOAD` can target an element set or one element. This makes small test decks compact while keeping large models readable through named sets.

2 Build and Command Line

FEMaster is built with CMake. A typical build produces an executable named `FEMaster` in the build output directory. The executable reads an input deck and writes a text result file. If no explicit output path is supplied, the result file uses the input base name with the extension `.res`.

2.1 Build

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --config Release
```

A release build is the normal choice for analysis work. A debug build is useful for small diagnostic models, assertions, and development checks, but it is usually too slow for larger finite-element models.

2.2 Running an input deck

```
FEMaster model.inp
FEMaster model.inp --output model.res
FEMaster model.inp --ncpus 8
```

Argument	Meaning
`input_file`	Path to the input deck. If no `.inp` suffix is supplied, it is added automatically.
`-output FILE`	Explicit result file path. Without this option, a `.res` file is derived from the input name.
`-ncpus N`	Number of threads allowed for parallel assembly and compute sections.

2.3 DSL documentation mode

The executable can print DSL registration information for the installed version. This is useful when checking which commands, keywords, and variants a particular build accepts.

```
FEMaster --doc-list
FEMaster --doc-show CLOAD
FEMaster --doc-variants ELASTIC
```

Option	Meaning
`-doc-list`	Print the available DSL commands.
`-doc-show CMD`	Show details for one command.
`-doc-tokens CMD`	Show expected tokens for a command.
`-doc-variants CMD`	Show accepted data-line variants.
`-doc-search TEXT`	Search command names and descriptions.
`-doc-where-token TOKEN`	Find commands that use a token.
`-doc-all`	Print the full command overview.

3 DSL Basics

3.1 Commands

A FEMaster deck consists of commands and data lines. A command starts with an asterisk, followed by the command name and optional comma-separated keywords.

Syntax

```
*COMMAND, KEY=value, FLAG
data, data, data
```

Command names and keywords are normalized by the parser. Case is not significant, and spaces in command names are removed before lookup. A consistent uppercase style is still recommended for readability.

3.2 Scopes

Some commands open an active context. `*MATERIAL` activates a material; subsequent material subcommands such as `*ELASTIC`, `*DENSITY`, and `*THERMALEXPANSION` apply to it. `*LOADCASE` activates an analysis context; commands such as `*SUPPORTS`, `*LOADS`, `*SOLVER`, and `*CONSTRAINTMETHOD` configure that loadcase.

No explicit end command is required. A context ends when a following command no longer belongs to it or when the file ends.

3.3 Aliases, defaults, and flags

Many keywords have aliases. `NAME` and `NSET` can identify node sets; `MATERIAL` and `MAT` are equivalent in sections. Defaults reduce boilerplate: `*NODE` defaults to `NALL`, `*ELEMENT` defaults to `EALL`, and `*SOLVER` defaults to `DEVICE=CPU` and `METHOD=DIRECT`.

Flags are keywords without a required value. `GENERATE` is the most common example. Explicit false-like values such as `NO`, `OFF`, `FALSE`, or `0` are interpreted as inactive.

3.4 Component order

Mechanical nodal vectors use `Ux, Uy, Uz, Rx, Ry, Rz`. The first three values are translations. The last three values are rotations. Loads, supports, velocities, accelerations, displacements, and many internal reductions use this convention.

3.5 Targets

A target can often be a set name or a single numeric ID. If a matching set name exists, the set is used. Otherwise FEMaster attempts to interpret the token as one ID.

4 Command Reference

This chapter gives a compact overview of the DSL commands. The following chapters describe the commands in more detail.

4.1 Geometry and topology

Command	Purpose
`*NODE`	Define nodes and coordinates.
`*ELEMENT`	Define elements by type and connectivity.
`*SURFACE`	Define element sides or element-set sides as surfaces.
`*NSET`	Define node sets.
`*ELSET`	Define element sets.
`*SFSET`	Define surface sets.

4.2 Properties

Command	Purpose
`*MATERIAL`	Create or activate a material.
`*ELASTIC`	Assign elastic material behaviour.
`*DENSITY`	Assign mass density.
`*THERMALEXPANSION`	Assign thermal expansion coefficient.
`*SOLIDSECTION`	Assign material and optional orientation to solid elements.
`*SHELLSECTION`	Assign material, thickness, and optional orientation to shell elements.
`*BEAMSECTION`	Assign material, profile, and local axis data to beam elements.
`*TRUSSSECTION`	Assign material and area to truss elements.
`*PROFILE`	Define beam profile constants.

4.3 Features, fields, loads, and constraints

Command	Purpose
`*POINTMASS`	Add concentrated mass, rotational inertia, and optional spring stiffness to node sets.
`*FIELD`	Define node, element, or integration-point data fields.
`*ORIENTATION`	Define a rectangular or cylindrical local coordinate system.
`*SUPPORT`	Add support entries to a support collector.

Command	Purpose
`*CLOAD`	Add concentrated nodal loads to a load collector.
`*DLOAD`	Add vector surface tractions to a load collector.
`*PLOAD`	Add pressure loads to a load collector.
`*VLOAD`	Add volume loads to a load collector.
`*TLOAD`	Add thermal loads from a temperature field.
`*INERTIALOAD`	Add rigid-body inertial loads.
`*AMPLITUDE`	Define a time function for transient load scaling.
`*COUPLING`	Define kinematic or structural couplings.
`*TIE`	Define tie constraints between slave nodes and master geometry.
`*CONNECTOR`	Define connector constraints between two node sets.
`*RBM`	Generate rigid-body-motion suppression equations.

4.4 Loadcase controls

Command	Purpose
`*LOADCASE`	Start an analysis block.
`*SUPPORTS`	Activate support collectors.
`*LOADS`	Activate load collectors.
`*SOLVER`	Select device and linear solution method.
`*CONSTRAINTMETHOD`	Select Nullspace or Lagrange constraint handling.
`*INERTIARELIEF`	Enable inertia relief for free static models.
`*REBALANCELOADS`	Balance residual resultant force and moment.
`*REQUESTSTIFFNESS`	Request stiffness matrix output.
`*REQUESTSTGEOM`	Request geometric stiffness matrix output.
`*NUMEIGENVALUES`	Set the number of eigenpairs.
`*SIGMA`	Set the buckling eigenvalue shift.
`*TOPODENSITY`	Select the topology density field.
`*TOPOORIENT`	Select the topology orientation field.
`*TOPOEXPONENT`	Set the topology penalization exponent.
`*TIME`	Set transient time range and step size.
`*NEWMARK`	Set Newmark integration parameters.
`*DAMPING`	Set Rayleigh damping.
`*WRITEEVERY`	Set transient output cadence.
`*INITIALVELOCITY`	Set transient initial velocity from a field.
`*CONSTRAINTSUMMARY`	Request generated-constraint diagnostics.
`*OVERVIEW`	Print a model overview.

5 Nodes

Nodes are the geometric reference points of a FEMaster model. Each node has an ID and three global coordinates. Elements, loads, supports, constraints, and features refer to nodes either directly or through node sets.

Syntax

```
*NODE, NSET=<optional-set>
<id>, <x>, <y>, <z>
```

`NSET` is optional. If omitted, nodes are added to `NALL`. Missing coordinate values are interpreted as zero, but production decks should normally write all three coordinates explicitly.

A node may have up to six mechanical degrees of freedom, `Ux, Uy, Uz, Rx, Ry, Rz`. Which of these are active depends on the connected elements and model objects. Solid elements usually need only translations. Shells, beams, point-mass springs, connectors, and constraints may require rotations.

5.1 Node IDs

Node IDs must be non-negative integers. The ID `0` is valid and can be used like any other node ID. Negative node IDs are not valid user-facing node identifiers in input decks because node IDs are used directly as row indices in nodal fields, connectivity, loads, supports, and result output.

FEMaster does not require a particular numbering strategy. Clear numbering is still useful for debugging, set construction, and result inspection. IDs should be unique and stable. If the same node ID is defined multiple times, the deck no longer gives a clear single coordinate for that node.

5.2 Coordinate meaning

Node coordinates are always global coordinates. Local coordinate systems do not change node positions; they only change how vector components are interpreted by commands such as `*SUPPORT`, `*CLOAD`, `*DLOAD`, `*VLOAD`, `*SOLIDSECTION`, `*SHELLSECTION`, and `*CONNECTOR`. This distinction matters: geometry is global, while loads, supports, material axes, and connector directions may be local.

5.3 Active degrees of freedom

FEMaster builds the active degree-of-freedom set from the model objects that reference each node. A node connected only to solid elements normally contributes three translational degrees of freedom. A shell or beam node contributes translations and rotations. A connector, coupling, support, or point-mass spring can also make rotational degrees of freedom relevant.

The presence of a node alone does not imply six active unknowns. The active set is determined when

the model is assembled. This keeps purely solid models smaller while still allowing mixed solid-shell-beam models to share the same nodal component convention.

6 Elements

Elements connect nodes and define the interpolation used over a line, surface, or volume region. An element alone does not define the complete physical model. A usable analysis also needs sections and materials for all element sets that contribute stiffness, mass, stress recovery, loads, or topology quantities.

Syntax

```
*ELEMENT, TYPE=<type>, ELSET=<optional-set>
<element-id>, <node-1>, <node-2>, ...
```

`TYPE` is required. `ELSET` is optional and defaults to `EALL`. Element IDs should be non-negative and unique for the same reason as node IDs: they are used as references by element sets, sections, loads, result fields, and diagnostics.

6.1 Registered element types

Type	Nodes	Family
`C3D4`	4	Linear tetrahedral solid.
`C3D5`	5	Pyramid input mapped to a degenerated hexahedral solid.
`C3D6`	6	Linear wedge solid.
`C3D8`	8	Linear hexahedral solid.
`C3D10`	10	Quadratic tetrahedral solid.
`C3D15`	15	Quadratic wedge solid.
`C3D20`	20	Quadratic hexahedral solid with full higher-order integration.
`C3D20R`	20	Quadratic hexahedral solid with reduced stiffness integration and higher-order mass integration.
`T3`	2	Two-node truss element.
`B33`	2 or 3	Two-node 3D beam element with optional orientation node.
`S3`	3	Three-node triangular shell.
`S4`	4	Four-node quadrilateral shell.
`MITC4`	4	Four-node quadrilateral shell with MITC-style tied shear interpolation.
`S6`	6	Six-node quadratic triangular shell.
`S8`	8	Eight-node quadratic quadrilateral shell.
`QSPT`	4	Specialized four-node quadrilateral shear-panel type.

6.2 Solid element family

Solid elements describe three-dimensional continua. They use translational nodal degrees of freedom and require `*SOLIDSECTION`. The section assigns material and optional material orientation. Solids are the

correct choice when the stress state is genuinely three-dimensional, when thickness assumptions are not acceptable, or when local load introduction, joints, or geometric details need volume resolution.

The main tradeoff is cost. A solid model usually needs more elements through thickness and more degrees of freedom than a shell or beam model. Low-order solid elements are robust and easy to mesh but may be too stiff in bending-dominated states. Higher-order solids improve curved geometry and strain representation but are more sensitive to element distortion and increase cost per element.

6.3 C3D4: 4-node tetrahedral solid

Syntax

```
*ELEMENT, TYPE=C3D4, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

C3D4 is a linear tetrahedral solid. It is the simplest volume element and is useful for complex geometry because tetrahedral meshes can fill almost any shape automatically. The element uses a tetrahedral reference domain and linear integration.

The primary advantage is meshing robustness. If the geometry is complicated, has many small details, or comes from an external mesher, C3D4 can often produce a valid mesh where structured hexahedral meshing would be difficult. It is also useful for quick checks and for regions where only rough stiffness representation is needed.

The main disadvantage is accuracy per degree of freedom. Linear tetrahedra are typically stiff in bending-dominated problems and can require many elements to reach acceptable accuracy. They also give constant strain over the element, so stress gradients are represented only through mesh refinement. For high-quality stress results, C3D10 is usually a better tetrahedral choice.

6.4 C3D5: 5-node pyramid input

Syntax

```
*ELEMENT, TYPE=C3D5, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <apex>
```

C3D5 accepts a five-node pyramid-style connectivity. FEMaster maps this input to a degenerated eight-node hexahedral topology by repeating the apex node in the upper hexahedral positions. It is therefore best understood as an import and transition convenience rather than as a separate high-quality pyramid formulation.

The advantage is compatibility with mixed meshes that contain pyramid transition elements between hexahedral and tetrahedral regions. It allows such meshes to be read without manually remeshing every transition.

The disadvantage is that degeneration can reduce element quality and may lead to less predictable stiffness and stress behaviour than a native well-shaped element. Use it deliberately, especially in regions with high stress gradients or bending. If pyramids dominate the structural response, validate the model against a known solution or consider remeshing.

6.5 C3D6: 6-node wedge solid

Syntax

```
*ELEMENT, TYPE=C3D6, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>
```

C3D6 is a linear wedge or prism solid. It has triangular faces at both ends and quadrilateral side faces. FEMaster uses wedge integration over an isoparametric wedge domain.

Wedges are useful for swept meshes, boundary-layer-like regions, and transition regions where triangular topology is natural in one plane but the model is extruded through thickness or along a path. Compared with tetrahedra, wedges can align better with layered geometry and can represent through-thickness stacks more cleanly.

The disadvantage is that linear wedges still have low-order displacement interpolation. They can be too stiff for bending if only a small number of elements is used through thickness. They also require reasonable prism quality; collapsed or highly skewed wedge elements can produce poor Jacobians and unreliable stresses.

6.6 C3D8: 8-node hexahedral solid

Syntax

```
*ELEMENT, TYPE=C3D8, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>
```

C3D8 is a linear hexahedral solid. FEMaster uses a hexahedral reference domain with quadratic-order integration for stiffness. Hexahedra are often the preferred low-order solid element when a good structured or swept mesh is available.

The main advantage is efficiency and accuracy for regular volume meshes. A well-shaped C3D8 mesh can represent many structural states more efficiently than a comparable linear tetrahedral mesh. Hexahedra are especially attractive for blocks, plates modelled with solids, extrusions, and regions where element directions should align with material or stress directions.

The main disadvantage is meshing difficulty. Poorly shaped hexahedra, warped elements, and strongly distorted Jacobians can degrade accuracy. Linear hexahedra can also be too stiff in bending if the mesh is coarse through thickness. For curved geometry or strong gradients, C3D20 or C3D20R may be more accurate.

6.7 C3D10: 10-node quadratic tetrahedral solid

Syntax

```
*ELEMENT, TYPE=C3D10, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>, <n5>, <n6>, <n7>, <n8>, <n9>, <n10>
```

C3D10 is a quadratic tetrahedral solid. It adds midside nodes to the linear tetrahedral topology and uses quadratic tetrahedral integration for stiffness. The mass integration uses a higher-order tetrahedral rule.

This is usually the better tetrahedral element for stress analysis and curved geometry. The quadratic interpolation captures bending and stress gradients much more effectively than C3D4. It is useful when

automatic tetrahedral meshing is needed but linear tetrahedra are too stiff or too inaccurate.

The disadvantages are larger element matrices, more nodes, and greater sensitivity to midside-node placement. Very distorted quadratic tetrahedra can behave poorly. They are also more expensive than `C3D4`, so coarse exploratory models may still use linear tetrahedra first.

6.8 `C3D15`: 15-node quadratic wedge solid

Syntax

```
*ELEMENT, TYPE=C3D15, ELSET=<set>
<id>, <n1>, ..., <n15>
```

`C3D15` is a quadratic wedge. It is the higher-order counterpart to `C3D6` and uses super-quadratic wedge integration. Its faces are quadratic triangles and quadratic quadrilaterals, which makes it suitable for swept meshes with curved boundaries or higher-order through-thickness interpolation.

The advantage is improved accuracy over linear wedges, especially in bending, curved geometry, and stress-gradient regions. It can be a strong choice for extruded triangular meshes where a hexahedral mesh is not available but a purely tetrahedral mesh is not desired.

The disadvantage is cost and sensitivity to geometry quality. Midside nodes must represent the intended curved or straight edges consistently. Poorly placed midside nodes can produce curved elements with bad Jacobians.

6.9 `C3D20`: 20-node quadratic hexahedral solid

Syntax

```
*ELEMENT, TYPE=C3D20, ELSET=<set>
<id>, <n1>, ..., <n20>
```

`C3D20` is a quadratic hexahedral solid. It uses a hexahedral reference domain with quartic-order integration for stiffness. It is intended for high-quality hexahedral meshes where accuracy per element is more important than minimal element cost.

The main advantage is high accuracy for smooth displacement and stress fields. Curved geometry and bending-dominated states are represented better than with `C3D8`. For well-shaped meshes, `C3D20` can deliver accurate stresses with fewer elements than a low-order mesh.

The main disadvantages are memory, compute cost, and mesh-quality sensitivity. A 20-node element has many more degrees of freedom than an 8-node element, and full higher-order integration increases cost. It is not a cure for poor mesh quality; distorted quadratic hexahedra can be worse than simpler well-shaped elements.

6.10 C3D20R: 20-node quadratic hexahedral solid with reduced integration

Syntax

```
*ELEMENT, TYPE=C3D20R, ELSET=<set>
<id>, <n1>, ..., <n20>
```

C3D20R uses the same quadratic hexahedral interpolation as C3D20, but uses reduced stiffness integration. In FEMaster the stiffness integration uses a lower hexahedral quadrature order than C3D20, while mass integration still uses a higher-order rule.

The advantage is lower stiffness-assembly cost and often better behaviour in bending or nearly incompressible situations where fully integrated elements may become overly stiff. Reduced integration is a common way to reduce locking tendencies and improve efficiency.

The disadvantage is that reduced integration can introduce spurious zero-energy or hourglass-like modes if the formulation and mesh do not control them sufficiently. It should be used with engineering judgement. Check displacement shapes, reactions, and stress fields, especially for coarse meshes or highly distorted elements.

6.11 Shell element family

Shell elements describe thin-walled structures with translational and rotational degrees of freedom. They require *SHELLSECTION. The section supplies material, thickness, optional orientation, and shear stiffness data. Shells are efficient when one dimension is much smaller than the other two and through-thickness stress details are not the primary modelling target.

The shell implementation uses local element axes, membrane strain, bending curvature, transverse shear strain, and section stiffness. It supports shell stress surfaces and shell resultants for the standard shell elements. Shells are efficient but depend strongly on element quality, normal direction, and appropriate thickness modelling.

6.12 S3: 3-node triangular shell

Syntax

```
*ELEMENT, TYPE=S3, ELSET=<set>
<id>, <n1>, <n2>, <n3>
```

S3 is a triangular shell element using the triangular reference domain with cubic-order integration. It is useful for meshing complex shell surfaces where quadrilateral elements are hard to generate.

The advantage is geometric flexibility. Triangles can fill arbitrary surfaces and transition regions easily. S3 is useful for imported shell meshes and for local geometric details.

The disadvantage is that triangular shell meshes can be less accurate and stiffer than good quadrilateral shell meshes for bending-dominated problems. Stress and curvature representation may require significant refinement. Use triangles where necessary, but prefer high-quality quadrilateral shells in regular regions.

6.13 S4: 4-node quadrilateral shell

Syntax

```
*ELEMENT, TYPE=S4, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

S4 is a four-node quadrilateral shell using a quadrilateral reference domain with cubic-order integration. It is the standard shell choice for many thin-walled structures when a regular quadrilateral mesh is available.

The advantage is efficiency and a natural fit for plates, panels, skins, webs, and other structured shell regions. With good aspect ratio and limited warping, S4 is a practical default for many shell models.

The disadvantage is sensitivity to shear locking, distortion, and warping in some bending-dominated cases. If transverse shear behaviour looks too stiff or the mesh is coarse, MITC4 may be preferable.

6.14 MITC4: 4-node quadrilateral shell with tied shear

Syntax

```
*ELEMENT, TYPE=MITC4, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

MITC4 has the same four-node quadrilateral topology as S4, but modifies the transverse shear interpolation. FEMaster ties shear strains at edge midpoints and interpolates them back into the element. Conceptually, this is used to reduce shear locking in thin-shell bending.

The advantage is improved bending behaviour for thin shells compared with a straightforward low-order shear interpolation. It is often the safer four-node quadrilateral shell when the structure is thin, bending-dominated, or when shear locking is suspected.

The disadvantage is that it is still a low-order quadrilateral. It does not remove the need for good mesh quality. Strong warping, poor aspect ratios, or irregular meshes can still reduce accuracy. For smooth geometry with higher-order meshing, S8 can be more accurate.

6.15 S6: 6-node quadratic triangular shell

Syntax

```
*ELEMENT, TYPE=S6, ELSET=<set>
<id>, <n1>, ..., <n6>
```

S6 is a quadratic triangular shell using cubic-order triangular integration. It adds midside nodes to the triangular shell topology.

The advantage is improved representation of curved shell geometry and smoother displacement/strain fields compared with S3. It is useful when triangular shell meshing is required but low-order triangles are too stiff or too coarse for stress recovery.

The disadvantage is higher cost and stronger dependence on midside-node quality. It can still be less efficient than a good quadrilateral shell mesh in regular areas.

6.16 S8: 8-node quadratic quadrilateral shell

Syntax

```
*ELEMENT, TYPE=S8, ELSET=<set>
<id>, <n1>, ..., <n8>
```

S8 is a quadratic quadrilateral shell using quintic-order quadrilateral integration. It is the high-order quadrilateral shell option.

The advantage is high accuracy for smooth shell fields, curved boundaries, and bending-dominated behaviour. It can reduce the number of elements needed to represent smooth curvature and stress gradients compared with S4.

The disadvantage is cost and mesh requirements. Midside-node placement matters, and distorted higher-order quadrilaterals can produce unreliable results. It should be used where the mesh quality justifies the higher-order interpolation.

6.17 QSPT: quadrilateral shear-panel type

Syntax

```
*ELEMENT, TYPE=QSPT, ELSET=<set>
<id>, <n1>, <n2>, <n3>, <n4>
```

QSPT is a specialized four-node quadrilateral panel-type element. It uses translational degrees of freedom only and focuses on shear-panel behaviour, shear flow, effective shear modulus, and related result extraction. It is not a general replacement for S4 or MITC4.

The advantage is that it can model panel-like shear transfer with a compact formulation and can provide shear-flow-oriented outputs. This can be useful for aircraft-style panels, webs, or simplified thin structures where shear flow is the main quantity of interest.

The disadvantage is limited generality. It does not carry the same full shell kinematics as the standard shell elements and should not be used when bending, drilling rotations, or general shell resultants are the modelling focus. Use it only when the intended panel formulation matches the engineering idealization.

6.18 Beam and truss element family

Line elements are efficient for slender structures. They reduce a three-dimensional member to a line with section properties. This is very efficient, but it also means that local 3D effects at joints, load introductions, holes, and connection details are not resolved unless additional modelling detail is added.

6.19 B33: 2-node 3D beam

Syntax

```
*ELEMENT, TYPE=B33, ELSET=<set>
<id>, <n1>, <n2>, <orientation-node>
```

`B33` is a two-node 3D beam. The third connectivity entry is optional and defines an orientation node for the local beam axis system. The beam uses six degrees of freedom per node and a beam section with a profile. The stiffness includes axial, bending, and torsional terms, profile inertias, product inertia handling through principal-axis rotation, and section offsets.

The advantage is efficiency for slender members. A frame, stiffener, spar cap, rod-like reinforcement, or beam idealization can be represented with very few degrees of freedom compared with a solid mesh. The element also supports local section forces, which are often more useful for beam design than raw 3D stresses.

The disadvantage is the beam assumption itself. Cross-section deformation, local stress concentrations, joint flexibility, and detailed load introduction are not resolved. The local axis definition must be robust: the orientation direction must not be parallel to the beam axis. If the member is short, thick, highly tapered, or has complex local connection behaviour, a shell or solid model may be more appropriate.

6.20 `T3`: 2-node truss

Syntax

```
*ELEMENT, TYPE=T3, ELSET=<set>
<id>, <n1>, <n2>
```

`T3` is a two-node truss element. It uses translational degrees of freedom only and carries axial force along its line direction. It requires `*TRUSSSECTION`, which supplies material and cross-sectional area.

The advantage is simplicity and efficiency. Trusses are appropriate for pin-jointed bars, rods, links, and idealized members that should carry only axial tension or compression. They are also useful for quick stiffness studies and simplified support structures.

The disadvantage is that a truss has no bending stiffness, no torsional stiffness, and no rotational degrees of freedom. It cannot represent frame action or moment transfer. Stress and strain recovery is intentionally limited; section-force output is generally the more useful result.

6.21 Choosing an element

Use solids when the stress state is three-dimensional or when local geometry matters. Use shells for thin-walled structures where thickness can be represented by a section. Use beams for slender members where section forces are the natural design quantities. Use trusses only when axial force is the intended physical behaviour.

For solids, prefer hexahedra when a good mesh is available. Use tetrahedra for complex geometry, preferably `C3D10` for serious stress work. Use reduced-integration elements such as `C3D20R` deliberately and check for spurious modes. For shells, prefer quadrilaterals in regular regions and triangles where topology requires them. For thin bending-dominated quadrilateral shells, `MITC4` is often a better low-order choice than `S4`.

7 Surfaces

Surfaces describe loadable or connectable element sides. They are used for pressure loads, distributed surface loads, tie constraints, and surface couplings. A surface is a selection of element sides; it is not a material or section property.

7.1 Surface from one element side

Syntax

```
*SURFACE, NAME=<surface-set>
<surface-id>, <element-id>, <side>
```

The side can be written as a number or as a token such as `S1` or `S2`. For shells, `SPOS` and `SNEG` select the positive or negative shell side.

7.2 Surface from an element set

Syntax

```
*SURFACE, NAME=<surface-set>, TYPE=ELEMENT
<element-set-or-id>, <side>
```

If the target is an element set, FEMaster creates surfaces for the selected side of all elements in the set. Otherwise the target is interpreted as a single element ID.

7.3 Orientation and sign

Surface loads depend on the chosen side and its normal direction. If pressure or traction direction is unexpected, check the selected side, element orientation, and shell side. For shells, positive and negative sides share the same geometric midsurface but have opposite normals.

8 Sets

Sets are named collections of IDs. They are central to readable FEMaster decks because most higher-level commands can target sets instead of repeating individual IDs.

8.1 Node sets

Syntax

```
*NSET, NAME=<name>, GENERATE
<id-start>, <id-end>, <increment>
```

Without `GENERATE`, data lines are read as explicit node IDs. Node sets are used by supports, concentrated loads, couplings, connectors, point masses, and many diagnostic workflows.

8.2 Element sets

Syntax

```
*ELSET, NAME=<name>, GENERATE
<id-start>, <id-end>, <increment>
```

Element sets are used by sections, volume loads, inertial loads, RBM constraints, and topology fields. One element may belong to several sets.

8.3 Surface sets

Syntax

```
*SFSET, NAME=<name>, GENERATE
<id-start>, <id-end>, <increment>
```

Surface sets group surface IDs. They are typically used by `*PLOAD`, `*DLOAD`, `*TIE`, and surface couplings.

8.4 Naming

Sets should be named by function, not only by geometry. Names such as `FIXED`, `LOAD_INTRODUCTION`, `BOLT_SURFACES`, or `SHELL_SKIN` remain understandable when mesh IDs change.

9 Sections and Profiles

Sections connect element sets to material and cross-section information. Without a matching section, an element may exist geometrically but cannot contribute the intended physical stiffness, mass, or stress recovery.

9.1 Solid section

Syntax

```
*SOLIDSECTION, ELSET=<elset>, MATERIAL=<material>, ORIENTATION=<orientation>
```

A solid section assigns a material to a volume element set. `ORIENTATION` is optional and is mainly relevant for anisotropic or orthotropic materials.

9.2 Shell section

Syntax

```
*SHELLSECTION, ELSET=<elset>, MATERIAL=<material>, THICKNESS=<t>, ORIENTATION=<orientation>
```

A shell section assigns material and thickness. Thickness affects membrane, bending, transverse shear, mass, and stress/resultant recovery. For anisotropic or ABD materials, orientation is part of the physical definition.

9.3 Beam profiles and beam sections

Syntax

```
*PROFILE, NAME=<name>
<A>, <Iy>, <Iz>, <J>, <Iyz>, <y0>, <z0>, ...
```

Syntax

```
*BEAMSECTION, ELSET=<elset>, MATERIAL=<material>, PROFILE=<profile>
<nx>, <ny>, <nz>
```

A profile stores beam cross-section constants. A beam section assigns profile and material to an element set. The optional direction line defines a local section direction and must not be parallel to the beam axis.

9.4 Truss section

Syntax

```
*TRUSSSECTION, ELSET=<elset>, MATERIAL=<material>, AREA=<A>
```

Truss elements use axial stiffness only. Bending, torsion, and rotational stiffness are not represented by truss elements.

10 Features

Features are additional model objects that are not defined through element connectivity. They add concentrated properties or auxiliary behaviour to the model.

10.1 Point Mass

Syntax

```
*POINTMASS, NSET=<node-set>
<m>, <Ixx>, <Iyy>, <Izz>, <Kx>, <Ky>, <Kz>, <Krx>, <Kry>, <Krz>
```

`*POINTMASS` assigns concentrated mass, rotational inertia, and optional spring stiffness to the nodes of a node set. It is useful for equipment masses, simplified component masses, sensors, local inertias, or concentrated replacement springs.

The data line contains four groups: scalar mass, three rotational inertias, three translational spring stiffnesses, and three rotational spring stiffnesses. Missing values are interpreted as zero.

Point masses can contribute to mass and inertia. They are therefore important for eigenfrequency analysis, transient analysis, inertial loads, and inertia relief. In a purely static loadcase without inertia relief, mass does not automatically mean weight; gravity or acceleration must be introduced through loads.

`*INERTIALOAD` and `*INERTIARELIEF` use the option `CONSIDER_POINT_MASSES` to decide whether point-mass features are included in the mass and inertia balance.

11 Material Models

Material definitions provide density, thermal expansion, and elastic constitutive behaviour. Sections reference materials by name; elements receive their actual stiffness and mass only after their section has been assigned. A material can therefore be defined once and reused by several shell, solid, beam, or truss sections.

11.1 Material context: *MATERIAL

Syntax

```
*MATERIAL, NAME=<material>
  *ELASTIC, TYPE=<model>
  ...
  *DENSITY
  ...
  *THERMALEXPANSION
  ...
```

*MATERIAL activates or creates a named material entry. The material name is supplied through MATERIAL or the alias NAME. Subcommands such as *ELASTIC, *DENSITY, and *THERMALEXPANSION apply to the currently active material.

11.2 Density: *DENSITY

Syntax

```
*DENSITY
<rho>
```

*DENSITY sets the mass density ρ. Density is used when FEMaster builds mass matrices, lumped masses from continuum or structural elements, inertia loads, inertia relief mass properties, and dynamic equations. In a consistent unit system, ρ must match the length, force, and time units used in the deck. For example, if length is in meters and force in Newtons, density is normally given in kg/m^3.

For a continuum element, the mass contribution is based on

$$M_e = \int_{\Omega_e} \rho N^T N \, d\Omega$$

or on the corresponding lumped form used by the selected analysis. For shells and beams, the section thickness or section area converts density into mass per area or mass per length.

11.3 Thermal expansion: *THERMALEXPANSION

Syntax

```
*THERMALEXPANSION
<alpha>
```

*THERMALEXPANSION sets the scalar thermal expansion coefficient α. Thermal loads use this coefficient together with the temperature difference $T - T_0$. For isotropic expansion, the thermal strain vector is

$$\varepsilon_{\text{th}} = \alpha(T - T_0)\begin{bmatrix}1 & 1 & 1 & 0 & 0 & 0\end{bmatrix}^T.$$

If a material has no thermal expansion coefficient, a thermal load can still be present in the deck, but it has no meaningful elastic thermal strain contribution for that material.

11.4 Isotropic elasticity

Syntax

```
*ELASTIC, TYPE=ISOTROPIC
<E>, <nu>
```

Accepted type values are ISO and ISOTROPIC. The data line contains Young's modulus E and Poisson's ratio ν. The shear modulus is not entered independently; it is derived as

$$G = \frac{E}{2(1+\nu)}.$$

For three-dimensional stress states, FEMaster uses the isotropic stiffness matrix

$$D = \frac{E}{(1+\nu)(1-2\nu)}\begin{bmatrix}1-\nu & \nu & \nu & 0 & 0 & 0\\ \nu & 1-\nu & \nu & 0 & 0 & 0\\ \nu & \nu & 1-\nu & 0 & 0 & 0\\ 0 & 0 & 0 & (1-2\nu)/2 & 0 & 0\\ 0 & 0 & 0 & 0 & (1-2\nu)/2 & 0\\ 0 & 0 & 0 & 0 & 0 & (1-2\nu)/2\end{bmatrix}.$$

For two-dimensional plane-stress-style element formulations, the in-plane matrix is

$$D_{2D} = \frac{E}{1-\nu^2}\begin{bmatrix}1 & \nu & 0\\ \nu & 1 & 0\\ 0 & 0 & (1-\nu)/2\end{bmatrix}.$$

11.5 Generalised isotropic elasticity

Syntax

```
*ELASTIC, TYPE=GENISO
<E>, <nu>, <G>
```

Accepted type values are `GENERALISEDISOTROPIC`, `GENERALISED_ISOTROPIC`, and `GENISO`. This model keeps the normal isotropic stiffness terms from E and ν, but uses the supplied G as an independent shear stiffness. In 3D, the normal block is identical to the isotropic material, while the shear diagonal entries are

$$D_{44} = D_{55} = D_{66} = G.$$

In the 2D matrix, the shear entry is also set directly:

$$D_{2D,33} = G.$$

This material is intentionally not a physically general isotropic law when $G \neq E/(2(1+\nu))$. It breaks the isotropic relationship between Young's modulus, Poisson's ratio, and shear modulus. That can be useful deliberately, especially for shell studies where the user wants to tune, reduce, or effectively ignore shear stiffness while keeping isotropic membrane and bending terms. It should not be used as a physical isotropic material unless the supplied G satisfies the isotropic relation.

11.6 Orthotropic elasticity

Syntax

```
*ELASTIC, TYPE=ORTHOTROPIC
<E1>, <E2>, <E3>, <G23>, <G13>, <G12>, <nu23>, <nu13>, <nu12>
```

Accepted type values are `ORTHO` and `ORTHOTROPIC`. Orthotropic elasticity defines three material axes with separate Young's moduli E_1, E_2, E_3, shear moduli G_{23}, G_{13}, G_{12}, and Poisson ratios. It is appropriate for laminates, wood-like materials, unidirectional composites, and any material whose stiffness depends on direction.

The 3D compliance matrix is assembled as

$$S = \begin{bmatrix} 1/E_1 & -\nu_{21}/E_2 & -\nu_{31}/E_3 & 0 & 0 & 0 \\ -\nu_{12}/E_1 & 1/E_2 & -\nu_{32}/E_3 & 0 & 0 & 0 \\ -\nu_{13}/E_1 & -\nu_{23}/E_2 & 1/E_3 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1/G_{23} & 0 & 0 \\ 0 & 0 & 0 & 0 & 1/G_{13} & 0 \\ 0 & 0 & 0 & 0 & 0 & 1/G_{12} \end{bmatrix}, \qquad D = S^{-1}.$$

Reciprocal Poisson ratios are derived from modulus ratios. In particular, FEMaster derives

$$\nu_{31} = \nu_{13}\frac{E_3}{E_1}.$$

The input column is named `NU13`, but the material construction uses the reciprocal value needed by the compliance matrix. Orthotropic data should be checked carefully because inconsistent modulus and Poisson-ratio combinations can produce a non-positive or ill-conditioned stiffness matrix.

11.7 ABD elasticity

Syntax

```
*ELASTIC, TYPE=ABD
<abd11>, <abd12>, ..., <abd66>,
<s11>, <s12>, <s21>, <s22>
```

`*ELASTIC, TYPE=ABD` reads 40 values. The first 36 values are a 6×6 ABD matrix in row-major order. The last 4 values are a 2×2 transverse shear matrix in row-major order. The values may be distributed over up to eight input lines.

The ABD matrix is a shell or laminate stiffness representation:

$$\begin{bmatrix} N \\ M \end{bmatrix} = \begin{bmatrix} A & B \\ B & D \end{bmatrix} \begin{bmatrix} \varepsilon_0 \\ \kappa \end{bmatrix}.$$

A is membrane stiffness, B is membrane-bending coupling, and D is bending stiffness. This model is appropriate when the section stiffness has already been homogenized outside FEMaster or when a laminate should be represented directly by engineering stiffness resultants rather than by individual plies.

Example

```
*MATERIAL, NAME=STEEL
*ELASTIC, TYPE=ISOTROPIC
210000000000.0, 0.3
*DENSITY
7850.0
*THERMALEXPANSION
1.2e-5
```

12 Coordinate Systems and Orientations

`*ORIENTATION` defines named local coordinate systems. These names can be referenced by sections, loads, supports, and connectors. The purpose is always the same: values that are convenient or physically meaningful in a local basis are transformed into the global basis used by the assembled finite-element equations.

Let the local basis vectors be the columns of a rotation matrix

$$R = \begin{bmatrix} e_1 & e_2 & e_3 \end{bmatrix}.$$

A local vector v_ℓ is transformed to global coordinates by

$$v_g = R v_\ell.$$

For displacements, forces, rotations, and moments, the same directional transformation is used for the three-component translational block and the three-component rotational block.

12.1 Rectangular systems

Syntax

```
*ORIENTATION, NAME=<name>, TYPE=RECTANGULAR
<x1>, <x2>, <x3>, <y1>, <y2>, <y3>, <z1>, <z2>, <z3>
```

`RECTANGULAR` describes a Cartesian local basis. The command accepts three practical input forms:

- 9 values: three direction vectors are supplied.
- 6 values: two direction vectors are supplied and the third is completed.
- 3 values: one primary direction is supplied and the remaining directions are completed.

The safest input is two or three non-collinear vectors. FEMaster normalizes and completes the basis, but it cannot infer a robust local frame from ambiguous geometry. If two supplied vectors are almost parallel, the cross product becomes small and the resulting local axes may be unstable. In formula form, a typical completion from two vectors a and b is

$$e_1 = \frac{a}{\|a\|}, \qquad e_3 = \frac{e_1 \times b}{\|e_1 \times b\|}, \qquad e_2 = e_3 \times e_1.$$

The exact input variant decides which vector is treated as primary, but the modelling implication is the same: provide directions that define a clear right-handed frame.

12.2 Cylindrical systems

Syntax

```
*ORIENTATION, NAME=<name>, TYPE=CYLINDRICAL
<origin-x>, <origin-y>, <origin-z>,
<axis-x>, <axis-y>, <axis-z>,
<reference-x>, <reference-y>, <reference-z>
```

`CYLINDRICAL` defines a coordinate system from an origin, an axis direction, and a reference direction. The axial unit vector is

$$e_z = \frac{a}{\|a\|}.$$

At a point x, the radial direction is obtained by projecting $x - o$ onto the plane normal to e_z:

$$r = (x - o) - ((x - o) \cdot e_z)e_z, \qquad e_r = \frac{r}{\|r\|}.$$

The tangential direction follows from

$$e_\theta = e_z \times e_r.$$

Near the cylindrical axis, the radial direction is not well defined. The reference vector is used to establish a stable angular reference, but users should still avoid applying cylindrical local directions exactly on the axis unless the intended direction is unambiguous.

12.3 Where orientations are used

Command	Use of the orientation
`*SUPPORT`	Prescribed support components are interpreted in the local basis before they become constraint equations.
`*CLOAD`	Nodal force and moment components are entered locally and transformed to global generalized forces.
`*DLOAD`	Surface traction components are interpreted in the local basis.
`*VLOAD`	Body-force components are interpreted in the local basis.
`*SOLIDSECTION`	Optional material axes for solid elements.
`*SHELLSECTION`	Optional shell material/resultant axes.
`*CONNECTOR`	Local connector axes define which relative translations or rotations are constrained.

Near-collinear directions

Coordinate systems, beam sections, couplings, and connectors are sensitive to near-collinear direction definitions. A nearly zero cross product creates poorly defined axes and can produce unintuitive constraints or stiffness directions. Prefer clear, normalized, geometrically separated reference directions.

13 Fields

*FIELD defines generic tabular data. Fields may be defined at root level or inside a loadcase block, depending on their intended use.

Syntax

```
*FIELD, NAME=<name>, DOMAIN=NODE|ELEMENT|IP, COLS=<n>, FILL=ZERO|NAN
<id>, <v1>, <v2>, ...
```

COLS is required and must be greater than zero. FEMaster limits fields to 64 components. FILL defaults to ZERO; NAN initializes unspecified values with NaN.

Domain	Meaning
NODE	One row per node ID. Used for temperature fields, initial velocities, or prescribed nodal data.
ELEMENT	One row per element ID. Used for topology density and orientation fields.
IP	Integration-point field. Used for data that is tied to element integration points.

Data lines may omit individual values; omitted values do not overwrite existing entries. The tokens NAN, INF, +INF, and -INF are accepted.

13.1 Typical references

Use	Domain/Cols	Referencing command
Temperature load	NODE, 1	*TLOAD with TEMPERATUREFIELD.
Initial velocity	NODE, 6	*INITIALVELOCITY.
Topology density	ELEMENT, 1	*TOPODENSITY.
Topology orientation	ELEMENT, 3	*TOPOORIENT.

14 Loads, Supports, and Amplitudes

Loads and supports are not activated directly by their definition command. They are assigned to named collectors first. A support collector is a named list of support entries. A load collector is a named list of load entries. A loadcase later selects one or more of these collectors with `*SUPPORTS` and `*LOADS`.

This collector model is important for reusable decks. For example, one support collector may contain all fixture supports, another may contain symmetry supports, and a third may contain temporary stabilization supports. A static loadcase can activate only the fixture collector, while a modal loadcase can activate fixture and symmetry collectors. Loads work the same way: each `*CLOAD`, `*DLOAD`, `*PLOAD`, `*VLOAD`, `*TLOAD`, or `*INERTIALOAD` entry declares which `LOAD_COLLECTOR` it belongs to; the loadcase decides which collectors are assembled.

14.1 Support collectors

Syntax

```
*SUPPORT, SUPPORT_COLLECTOR=<name>, ORIENTATION=<orientation>
<node-set-or-id>, <ux>, <uy>, <uz>, <rx>, <ry>, <rz>
```

`*SUPPORT` adds one support entry to the collector named by `SUPPORT_COLLECTOR`. The collector is created implicitly when the first support uses its name. The data line targets either a node set or a single node id and then gives prescribed values for the six nodal components `Ux, Uy, Uz, Rx, Ry, Rz`. In most models the prescribed values are zero, but nonzero entries can be used for enforced displacement or rotation.

If `ORIENTATION` is omitted, the six values are interpreted in the global coordinate system. If an orientation is given, the translational and rotational components are interpreted in that local coordinate system and transformed into the global system before the constraint equations are built. This is the usual way to describe supports normal to a cylindrical surface, sliding conditions in a skew system, or fixtures that are easier to express in local axes than in global axes.

The support value is a kinematic prescription. In matrix form, a scalar support can be read as

$$e_i^T u = \bar{u}_i,$$

where e_i selects the constrained degree of freedom and $\bar{u}_i$ is the prescribed value from the support line. Multiple support entries in the same collector are assembled into one set of equations.

14.2 Concentrated loads

Syntax

```
*CLOAD, LOAD_COLLECTOR=<name>, ORIENTATION=<orientation>, AMPLITUDE=<amp>
<node-set-or-id>, <Fx>, <Fy>, <Fz>, <Mx>, <My>, <Mz>
```

`*CLOAD` adds nodal forces and moments to the collector named by `LOAD_COLLECTOR`. The target may be a node set or a single node id. The six numeric values correspond to nodal generalized force components conjugate to `Ux, Uy, Uz, Rx, Ry, Rz`. Translational components are forces; rotational components are moments.

If an orientation is specified, the force and moment components are interpreted in the local system and transformed to global components during assembly. If an amplitude is specified, the load entry is multiplied by the amplitude value in transient analyses:

$$f_j(t) = a(t) f_j.$$

In static analyses, the load is used as its defined value unless the loadcase explicitly evaluates time-dependent data.

14.3 Surface traction and pressure

Syntax

```
*DLOAD, LOAD_COLLECTOR=<name>, ORIENTATION=<orientation>, AMPLITUDE=<amp>
<surface-set-or-id>, <Fx>, <Fy>, <Fz>
```

Syntax

```
*PLOAD, LOAD_COLLECTOR=<name>, AMPLITUDE=<amp>
<surface-set-or-id>, <pressure>
```

`*DLOAD` defines a vector traction on surfaces. Conceptually, the load contribution is

$$f_e = \int_{\Gamma_e} N^T t \, d\Gamma,$$

where t is the traction vector and N contains the element shape functions on the loaded surface. `*PLOAD` defines a scalar pressure. A pressure is converted into a traction using the surface normal, so the selected surface side and its normal direction determine the sign convention:

$$t = pn.$$

Use `*DLOAD` when the traction direction is known as a vector. Use `*PLOAD` when the load should follow the surface normal. Both commands add entries to a load collector and are activated only when that collector is referenced by a loadcase.

14.4 Volume loads

Syntax

```
*VLOAD, LOAD_COLLECTOR=<name>, ORIENTATION=<orientation>, AMPLITUDE=<amp>
<element-set-or-id>, <Fx>, <Fy>, <Fz>
```

`*VLOAD` defines a body-force density over an element region. The assembled contribution follows

$$f_e = \int_{\Omega_e} N^T b \, d\Omega,$$

where b is the supplied volume load vector. This is appropriate for load densities that are already expressed per unit volume. If a gravitational acceleration should be converted into force, the material density and the intended unit convention must be reflected in the supplied values or in the load formulation used for the model.

14.5 Thermal loads

Syntax

```
*TLOAD, LOAD_COLLECTOR=<name>, TEMPERATUREFIELD=<field>, REFERENCETEMPERATURE=<T0>
```

`*TLOAD` adds thermal strain loading to a load collector. The command references a nodal temperature field and a reference temperature. For a material with thermal expansion coefficient α, the thermal strain is based on

$$\varepsilon_{\text{th}} = \alpha(T - T_0)\begin{bmatrix}1 & 1 & 1 & 0 & 0 & 0\end{bmatrix}^T$$

for isotropic expansion. The corresponding equivalent mechanical load is assembled from the element constitutive law and the thermal strain. Thermal loads therefore require both a temperature field and material definitions with thermal expansion data.

14.6 Inertial loads

Syntax

```
*INERTIALOAD, LOAD_COLLECTOR=<name>, CONSIDER_POINT_MASSES=0|1
<elset>, <cx>, <cy>, <cz>, <a0x>, <a0y>, <a0z>, <wx>, <wy>, <wz>, <alphax>, <alphay>, <alphaz
    >
```

`*INERTIALOAD` creates equivalent nodal loads from a prescribed rigid-body acceleration field. The first vector c is the reference point. The second vector a_0 is the translational acceleration of that point. The third vector ω is angular velocity and the fourth vector α is angular acceleration. At a material point x, the rigid-body acceleration is

$$a(x) = a_0 + \alpha \times (x - c) + \omega \times (\omega \times (x - c)) \,.$$

The equivalent inertial load is assembled over the selected element set. `CONSIDER_POINT_MASSES` controls whether point-mass features are included as additional concentrated inertia contributions.

14.7 Amplitudes

Syntax

```
*AMPLITUDE, NAME=<name>, TYPE=LINEAR|STEP|NEAREST
<time>, <value>
```

An amplitude is a scalar time function. Load entries that reference an amplitude are scaled by its value during transient analysis. `LINEAR` interpolates linearly between data points. `STEP` keeps the previous value until the next point is reached. `NEAREST` uses the value of the closest tabulated time. Amplitudes are useful when the spatial load pattern is fixed but the intensity changes over time.

15 Constraints

Constraints define kinematic equations between degrees of freedom. They are different from supports in modelling intent: a support prescribes motion of selected degrees of freedom, while a constraint relates several nodes, surfaces, or reference points to each other. FEMaster assembles supports and constraints into one algebraic equation set before the active loadcase is solved.

Most constraint equations can be written as

$$Cu = d.$$

u is the vector of active nodal degrees of freedom, C is the constraint matrix, and d is the right-hand side. Homogeneous constraints have $d = 0$. The constraint definitions in this chapter describe which equations are generated. The numerical handling of those equations, for example Nullspace reduction or Lagrange multipliers, is selected by `*CONSTRAINTMETHOD` and described in Chapter 16.

15.1 Rigid-body motion suppression: `*RBM`

Syntax

```
*RBM, ELSET=<elset>
```

`*RBM` generates up to six homogeneous equations for the nodes used by the selected element set. `ELSET` also accepts the alias `SET`; if omitted, the default is `EALL`. The command is meant to suppress free rigid-body modes without fixing an arbitrary node. This is especially useful for stabilization, diagnostic models, and inertia-relief workflows.

FEMaster first collects all nodes that belong to the selected structural elements. It computes the center of gravity x_0 of the selected element region and uses the relative position

$$r_i = x_i - x_0$$

for every participating node i. With n participating nodes and $w = 1/n$, the three translational equations are

$$\sum_i w u_{ix} = 0, \qquad \sum_i w u_{iy} = 0, \qquad \sum_i w u_{iz} = 0.$$

The three rotational equations suppress the average rigid rotation measured by the first moment of the displacement field:

$$\sum_i w(r_i \times u_i)_x = 0, \qquad \sum_i w(r_i \times u_i)_y = 0, \qquad \sum_i w(r_i \times u_i)_z = 0.$$

Expanded component-wise, these are

$$\sum_i w(r_{iy} u_{iz} - r_{iz} u_{iy}) = 0,$$

$$\sum_i w(r_{iz}u_{ix} - r_{ix}u_{iz}) = 0,$$

$$\sum_i w(r_{ix}u_{iy} - r_{iy}u_{ix}) = 0.$$

Only active translational degrees of freedom are used. If a required component does not exist for a node, it is skipped for that equation.

15.2 Coupling: *COUPLING

Syntax

```
*COUPLING, MASTER=<master-nset>, TYPE=KINEMATIC|STRUCTURAL, SLAVE=<slave-nset>
<Ux>, <Uy>, <Uz>, <Rx>, <Ry>, <Rz>
```

Syntax

```
*COUPLING, MASTER=<master-nset>, TYPE=KINEMATIC|STRUCTURAL, SFSET=<surface-set>
<Ux>, <Uy>, <Uz>, <Rx>, <Ry>, <Rz>
```

`*COUPLING` connects one master node set to either a slave node set or a slave surface set. Exactly one of `SLAVE` and `SFSET` must be given. In practical models the master set should normally contain one reference node. Larger master sets can over-constrain the model because the generated equations are based on a single master node.

The data line contains six flags for `Ux, Uy, Uz, Rx, Ry, Rz`. Values greater than zero activate the corresponding coupling component. `KINEMATIC` generates constraint equations. `STRUCTURAL` does not generate kinematic equations; it distributes generalized loads from the master node to slave nodes in a work-equivalent way.

15.2.1 Kinematic coupling equations

For a slave node s, master node m, and offset

$$r = x_s - x_m,$$

the translational kinematic relation is the small-rotation rigid-body relation

$$u_s = u_m + \theta_m \times r.$$

FEMaster writes this component-wise as homogeneous equations. For example,

$$u_{sx} - u_{mx} - r_z\theta_{my} + r_y\theta_{mz} = 0,$$

$$u_{sy} - u_{my} - r_x\theta_{mz} + r_z\theta_{mx} = 0,$$

$$u_{sz} - u_{mz} - r_y\theta_{mx} + r_x\theta_{my} = 0.$$

Rotational coupling is direct:

$$\theta_{sx} - \theta_{mx} = 0, \qquad \theta_{sy} - \theta_{my} = 0, \qquad \theta_{sz} - \theta_{mz} = 0.$$

Only the components enabled by the six flags are generated, and only if the involved degrees of freedom exist.

15.2.2 Structural coupling load distribution

For `STRUCTURAL`, a generalized master load

$$b = \begin{bmatrix} F_x & F_y & F_z & M_x & M_y & M_z \end{bmatrix}^T$$

is distributed to slave translational forces. FEMaster builds the equilibrium operator

$$A = \begin{bmatrix} I & I & \dots & I \\ [r_1]_\times & [r_2]_\times & \dots & [r_n]_\times \end{bmatrix},$$

where $[r_i]_\times f_i = r_i \times f_i$. The distributed force vector is computed in a least-norm, work-equivalent form from

$$G\lambda = b, \qquad G = AA^T, \qquad f = A^T\lambda.$$

The resulting slave forces reproduce the master resultant force and moment as closely as the slave geometry permits. The master load is then consumed because it has been represented by slave forces.

15.3 Tie: *TIE

Syntax

```
*TIE, MASTER=<master>, SLAVE=<slave>, ADJUST=NO|YES, DISTANCE=<search-distance>
```

`*TIE` binds slave nodes to a master surface set or master line set. The slave region may be a node set or a surface set. For each slave node, FEMaster searches the closest master geometry within `DISTANCE`. If no candidate is found, no equation is generated for that slave node. If `ADJUST=YES`, the slave node coordinate is moved to the projected master position before the equations are assembled.

Let s be a slave node projected onto a master surface or line at local coordinates ξ. Let $N_a(\xi)$ be the shape function of master node a at that projected point. For every active degree of freedom, the compatibility equation is

$$u_s - \sum_a N_a(\xi) u_a = 0.$$

This ties the slave degree of freedom to the interpolated master motion. For surface masters the interpolation uses the surface shape functions; for line masters it uses the line shape functions. The same equation form is used for translational and rotational degrees of freedom when those degrees of freedom exist on both slave and all master nodes.

15.4 Connector: *CONNECTOR

Syntax

```
*CONNECTOR, TYPE=<type>, NSET1=<nset1>, NSET2=<nset2>, COORDINATESYSTEM=<orientation>
```

`*CONNECTOR` creates idealized relative-motion constraints between two node sets in a named coordinate system. Supported types are `BEAM`, `HINGE`, `CYLINDRICAL`, `TRANSLATOR`, `JOIN`, and `JOINRX`. The connector type maps to a six-component mask that decides which local translations and rotations are constrained.

For each constrained local component e_α, FEMaster transforms the local direction into the global system:

$$g_\alpha = Re_\alpha.$$

For a constrained translation, the generated equation is

$$g_\alpha^T(u_1 - u_2) = 0.$$

For a constrained rotation, the equation is

$$g_\alpha^T(\theta_1 - \theta_2) = 0.$$

The connector therefore constrains relative motion along selected local axes rather than simply tying global components. This is why the referenced coordinate system is part of the connector definition and should be chosen carefully.

15.5 Constraint summary

Syntax

```
*LOADCASE, TYPE=LINEARSTATIC
  *CONSTRAINTSUMMARY
```

`*CONSTRAINTSUMMARY` requests a diagnostic listing of the internally generated constraint equations for the active loadcase. The output is grouped by constraint origin, for example supports, RBM equations, couplings, ties, and connectors. This is useful when a model is unexpectedly singular, over-constrained, or when a surface tie did not find as many slave projections as intended. It is a diagnostic report of generated equations, not a modelling command that changes the equations.

Master sets

Many coupling and connector workflows are intended for one reference node in the master set. Larger master sets may over-constrain the model or produce a kinematic interpretation that is different from the intended reference-point behaviour.

16 Analysis and Solver Controls

This chapter describes commands that are used inside loadcases to activate collectors, choose the linear solver, select the constraint backend, and configure analysis-specific numerical options. Constraint definitions themselves are described in Chapter 15; the commands here decide how those definitions are used in a particular analysis.

16.1 Activating support and load collectors

Syntax

```
*SUPPORTS
<support-collector-1>, <support-collector-2>, ...

*LOADS
<load-collector-1>, <load-collector-2>, ...
```

`*SUPPORTS` activates named support collectors that were populated by `*SUPPORT`. `*LOADS` activates named load collectors that were populated by `*CLOAD`, `*DLOAD`, `*PLOAD`, `*VLOAD`, `*TLOAD`, or `*INERTIALOAD`. The loadcase does not activate every support or load in the model automatically. It assembles only the collectors listed in these commands.

Mathematically, active support collectors contribute equations to the global constraint set $Cu = d$. Active load collectors contribute to the right-hand side vector f. In a transient loadcase, time-dependent amplitudes are evaluated during time integration, so the active load vector is a function $f(t)$.

16.2 Solver

Syntax

```
*SOLVER, DEVICE=CPU|GPU, METHOD=DIRECT|INDIRECT
```

`DEVICE` selects the execution device. `CPU` is the standard path. `GPU` is intended for configurations where the corresponding solver routines and data structures are available. `METHOD` selects the strategy for linear systems.

`DIRECT` solves a system by factorization. For a static reduced system

$$Aq = b,$$

the matrix A is factorized and the solution is obtained by triangular solves or an equivalent direct procedure. Direct solvers are robust and predictable for small and medium models. Their main limitation is memory growth, especially for large three-dimensional meshes.

`INDIRECT` uses an iterative solution path. Instead of building a full direct factorization, an approximate solution is improved until the residual is sufficiently small. Iterative methods can be attractive for large systems because they avoid some of the memory cost of direct factorization. They are more sensitive to conditioning, scaling, constraint handling, and matrix definiteness. In FEMaster the indirect path should be used with `NULLSPACE` constraints, because the reduced system keeps the standard structural form. It is not intended for the saddle-point system produced by Lagrange multipliers.

16.3 Constraint method

Syntax

```
*CONSTRAINTMETHOD, TYPE=NULLSPACE|LAGRANGE
```

The active supports and constraints define

$$Cu = d.$$

`*CONSTRAINTMETHOD` selects how this equation set is introduced into the analysis equations.

16.3.1 Nullspace method

`NULLSPACE` parameterizes all admissible displacements as

$$u = u_p + Tq.$$

u_p is a particular displacement vector satisfying the inhomogeneous part of the constraints. T is a basis for the nullspace of C, so

$$CT = 0, \qquad Cu_p = d.$$

q contains only independent degrees of freedom. A static system

$$Ku = f$$

is transformed into

$$T^T KTq = T^T(f - Ku_p).$$

The matrix

$$A = T^T KT$$

is the reduced stiffness matrix. This approach removes constrained degrees of freedom from the solve and is well suited for direct and iterative solvers. It also avoids adding Lagrange multipliers to the unknown vector.

For eigenvalue and transient problems the same projection is applied to the operators:

$$K_r = T^T KT, \qquad M_r = T^T MT, \qquad C_r = T^T CT.$$

Recovered displacements are mapped back with $u = u_p + Tq$, while velocities and accelerations use the homogeneous part $T\dot{q}$ and $T\ddot{q}$.

16.3.2 Lagrange method

`LAGRANGE` keeps the original displacement vector and adds one multiplier per constraint equation. The static system becomes

$$\begin{bmatrix} K & C^T \\ C & -\epsilon I \end{bmatrix} \begin{bmatrix} u \\ \lambda \end{bmatrix} = \begin{bmatrix} f \\ d \end{bmatrix}.$$

λ contains the Lagrange multipliers, which can be interpreted as generalized constraint reactions. ϵ is a possible small regularization term when configured by the solver path; conceptually, the pure Lagrange system has $\epsilon = 0$.

This formulation is direct and preserves the original displacement variables, but it creates a saddle-point problem. Such systems are indefinite and more demanding numerically. In FEMaster, `LAGRANGE` requires the direct linear solver for linear static analysis. Use `NULLSPACE` when an indirect solver is desired.

16.4 Inertia relief

Syntax

```
*INERTIARELIEF, CONSIDER_POINT_MASSES=0|1
```

Inertia relief is used for static analysis of free or nearly free structures. A free body under an unbalanced external load has no static equilibrium in an inertial frame; it accelerates as a rigid body. Inertia relief computes an equivalent rigid-body acceleration and adds inertia loads that balance the external resultant force and moment. The remaining static solve describes elastic deformation relative to that accelerating rigid-body motion.

This is not a substitute for missing real supports. If the physical model has bearings, bolts, fixtures, or symmetry conditions, they should be modelled as supports or constraints. Inertia relief is appropriate when the analysed body is intentionally free, for example an aircraft component under manoeuvre load or a structure in a load case where fixed ground supports would be artificial.

`CONSIDER_POINT_MASSES` controls whether point-mass features are included in the mass properties used by inertia relief. If point masses represent relevant equipment or lumped mass, set this option to 1. If they are purely numerical or should not affect the inertial balance, leave them out.

16.5 Load rebalancing

Syntax

```
*REBALANCELOADS
```

Load rebalancing modifies the external load balance so that the resultant force and moment vanish. It is a numerical balancing tool, not a physical rigid-body acceleration model. It is useful when a load is intended to be self-equilibrated but small residual resultants appear because of discretization, surface selection, rounding, or simplified load introduction.

The distinction from inertia relief is important. Inertia relief says: the structure is free, the external load would accelerate it, and inertia loads should represent that acceleration. Load rebalancing says: the load should be balanced, and the residual imbalance should be corrected. For real free-body operating loads, inertia relief is usually the more physical interpretation. For small unintended imbalance in an otherwise self-equilibrated load, rebalancing is often the simpler diagnostic option.

16.6 Matrix and diagnostic requests

Syntax

```
*REQUESTSTIFFNESS, FILE=<base-file>
*REQUESTSTGEOM, FILE=<base-file>
*CONSTRAINTSUMMARY
```

`*REQUESTSTIFFNESS` requests stiffness matrix output for diagnostics or external comparison. `*REQUESTSTGEOM` requests geometric stiffness output in buckling analysis. `*CONSTRAINTSUMMARY` prints the internally generated constraint equations grouped by origin, such as supports, RBM equations, couplings, ties, and connectors.

These commands should be treated as diagnostic output controls. They do not change the physical model. They are helpful when validating assemblies, comparing reduced matrices, debugging constraint generation, or checking why a loadcase is singular or over-constrained.

16.7 Eigenvalue parameters

Syntax

```
*NUMEIGENVALUES
<count>

*SIGMA
<shift>
```

`*NUMEIGENVALUES` sets how many eigenpairs are requested for eigenfrequency or buckling analysis. The value must be positive.

`*SIGMA` is used in linear buckling. The reduced buckling problem is assembled as

$$A\phi = \lambda(-B)\phi,$$

where

$$A = T^T K T, \qquad B = T^T K_G T.$$

The reported buckling factors are sorted ascending after the eigensolve. The shift σ guides the buckling eigenvalue search and is the smallest buckling factor region that should be targeted for output. It must be chosen carefully: a poor shift can make the eigensolver converge to an unintended part of the spectrum or miss the factors of engineering interest.

If `*SIGMA` is set to 0, FEMaster generates a shift automatically. It first tries a Rayleigh estimate from the pre-buckling displacement q_{pre}:

$$\lambda_{\text{raw}} = \frac{q_{\text{pre}}^T A q_{\text{pre}}}{q_{\text{pre}}^T (-B) q_{\text{pre}}}.$$

If that estimate is finite and positive, the generated shift is

$$\sigma = \frac{\lambda_{\text{raw}}}{10^6}.$$

If the Rayleigh estimate is not usable, FEMaster falls back to a diagonal ratio estimate

$$\lambda_i = \frac{A_{ii}}{(-B)_{ii}}$$

for positive finite diagonal entries and uses a conservative lower quartile of those values. If no valid estimate exists, the fallback shift is 10^{-12}.

16.8 Transient-specific controls

Syntax

```
*TIME
<t_start>, <t_end>, <dt>
```

`*TIME` defines the time interval and fixed time step for `LINEARTRANSIENT`. It accepts either three values $(t_{\text{start}}, t_{\text{end}}, \Delta t)$ or two values $(t_{\text{end}}, \Delta t)$, in which case the start time is 0. The number of integration steps is based on the interval length and Δt. A smaller time step gives better temporal resolution but increases the number of linear solves and result states.

Syntax

```
*NEWMARK
<beta>, <gamma>
```

`*NEWMARK` sets the Newmark integration parameters β_N and γ_N. The transient equation is

$$M\ddot{u} + C\dot{u} + Ku = f(t).$$

In reduced coordinates FEMaster solves the projected form

$$M_r\ddot{q} + C_r\dot{q} + K_r q = r(t).$$

The common values $\beta_N = 0.25$ and $\gamma_N = 0.5$ correspond to the average-acceleration method for linear systems. Other values should be chosen deliberately because they change numerical damping, stability, and accuracy.

Syntax

```
*DAMPING, TYPE=RAYLEIGH
<alpha>, <beta>
```

`*DAMPING` currently supports Rayleigh damping:

$$C = \alpha M + \beta K.$$

α controls mass-proportional damping and has stronger influence at low frequencies. β controls stiffness-proportional damping and has stronger influence at high frequencies. In reduced form FEMaster builds

$$C_r = \alpha M_r + \beta K_r.$$

Syntax

```
*WRITEEVERY, TYPE=STEPS|TIME
<value>
```

`*WRITEEVERY` controls transient result cadence. With `TYPE=STEPS`, the value is rounded to an integer and results are written every N steps. With `TYPE=TIME`, the value is a physical output interval; FEMaster converts it to a step stride by dividing by the analysis time step. The final time state is written even when it does not fall exactly on the stride.

Syntax

```
*INITIALVELOCITY, FIELD=<node-field>
```

`*INITIALVELOCITY` references a node field with six components. The field is reduced into the active coordinate space and used as the initial velocity. If no initial velocity is specified, the initial reduced velocity is zero. Initial acceleration is computed internally from the initial state, the active matrices, and the load at the start time.

17 Loadcases

A loadcase defines which analysis is executed on the previously defined model. The model describes what exists: nodes, elements, surfaces, sets, materials, sections, features, loads, supports, constraints, and fields. The loadcase describes which equation is assembled, which collectors are active, how constraints are handled, which solver is used, and which analysis-specific parameters are applied.

17.1 General syntax

Syntax

```
*LOADCASE, TYPE=<type>, NAME=<optional-name>
  <loadcase commands>
```

`TYPE` is required. `NAME` is optional and is used for readability in the input deck and result output. Loadcases are executed in input order. Result blocks are written in the same order.

17.2 Common equation structure

Linear loadcases use global matrices assembled from element contributions. The most common operators are stiffness K, mass M, damping C, and geometric stiffness K_G. Active load collectors produce a right-hand side f. Active supports and constraints produce equations

$$C_c u = d.$$

The selected constraint method then either reduces the problem with $u = u_p + Tq$ or augments it with Lagrange multipliers. The loadcase formulas below are written in their physical full-space form first; the actual solved system may be the reduced or augmented form described in Chapter 16.

17.3 Linear Static

`LINEARSTATIC` solves small-displacement linear static equilibrium. It is the standard analysis for displacements, reactions, strains, stresses, and static load paths.

17.3.1 Governing equation

The physical equation is

$$Ku = f.$$

After Nullspace constraint reduction, FEMaster solves

$$T^T KTq = T^T(f - Ku_p), \qquad u = u_p + Tq.$$

With Lagrange constraints, the static system is

$$\begin{bmatrix} K & C_c^T \\ C_c & 0 \end{bmatrix} \begin{bmatrix} u \\ \lambda \end{bmatrix} = \begin{bmatrix} f \\ d \end{bmatrix}$$

up to possible numerical regularization.

17.3.2 Typical DSL

Example

```
*LOADCASE, TYPE=LINEARSTATIC, NAME=static-load
  *SUPPORTS
  FIXED
  *LOADS
  FORCE
  *SOLVER, DEVICE=CPU, METHOD=DIRECT
  *CONSTRAINTMETHOD, TYPE=NULLSPACE
```

Common commands are `*SUPPORTS`, `*LOADS`, `*SOLVER`, and `*CONSTRAINTMETHOD`. Optional controls include `*INERTIARELIEF`, `*REBALANCELOADS`, `*REQUESTSTIFFNESS`, and `*CONSTRAINTSUMMARY`.

17.4 Linear Static Topology

`LINEARSTATICTOPO` solves a static problem whose element stiffness can be modified by topology fields. It is useful for topology studies, density-based parameter sweeps, and orientation-dependent stiffness experiments.

17.4.1 Governing equation

The equation is

$$K(\rho, \theta)u = f.$$

ρ is an element density-like field and θ is an element orientation-like field. The density field is typically used through a penalized stiffness contribution

$$K_e^{\text{eff}} = \rho_e^p K_e,$$

where p is supplied by `*TOPOEXPONENT`. Low densities reduce stiffness contribution; high densities approach the full element stiffness. The orientation field is used by element/material formulations that evaluate local orientation data.

17.4.2 Typical DSL

Syntax

```
*LOADCASE, TYPE=LINEARSTATICTOPO, NAME=<name>
  *TOPODENSITY
  <element-field>
  *TOPOORIENT
  <element-field>
  *TOPOEXPONENT
  <p>
```

`*TOPODENSITY` requires an element field with one component. `*TOPOORIENT` requires an element field with three components. The static controls from `LINEARSTATIC` are used in the same way.

The compliance, density sensitivity, and orientation sensitivity derivations are collected in Appendix A.

17.5 Eigenfrequency

`EIGENFREQ` computes natural frequencies and mode shapes of the linearized model. Loads are not used because the free-vibration problem is defined by stiffness, mass, and boundary conditions.

17.5.1 Governing equation

The full-space eigenproblem is

$$K\phi_i = \omega_i^2 M\phi_i.$$

ϕ_i is the mode shape and ω_i is the circular frequency. The frequency in Hertz is

$$f_i = \frac{\omega_i}{2\pi}.$$

With Nullspace constraints, the reduced problem is

$$T^T KT\psi_i = \omega_i^2 T^T MT\psi_i, \qquad \phi_i = T\psi_i.$$

17.5.2 Typical DSL

Syntax

```
*LOADCASE, TYPE=EIGENFREQ, NAME=<name>
  *SUPPORTS
  <support-collector>
  *NUMEIGENVALUES
  <count>
```

A free-free model can contain rigid-body modes with zero or near-zero frequency. A supported model computes modes relative to the active supports and constraints.

17.6 Linear Buckling

`LINEARBUCKLING` computes ideal linear buckling factors and buckling modes around a preloaded reference state. It first solves a static preload problem, then assembles geometric stiffness from the resulting stresses, and finally solves a generalized eigenvalue problem.

17.6.1 Governing equations

The preload solve is

$$Ku_0 = f_{\text{pre}}.$$

With constraints, FEMaster solves the reduced preload equation

$$Aq_{\text{pre}} = b, \qquad A = T^T KT, \qquad b = T^T(f_{\text{pre}} - Ku_p).$$

From the preload stresses, FEMaster builds the geometric stiffness K_G and reduces it to

$$B = T^T K_G T.$$

The buckling eigenproblem is solved as

$$A\phi_i = \lambda_i(-B)\phi_i.$$

λ_i is the buckling factor. For the preload pattern f_{pre}, the ideal linear critical load for mode i is

$$f_{\text{crit},i} = \lambda_i f_{\text{pre}}.$$

17.6.2 Sigma

`*SIGMA` controls the buckling eigenvalue shift. It should be set carefully because it guides the part of the spectrum returned by the eigensolver. If `*SIGMA` is zero, FEMaster estimates a shift automatically from the preload displacement with the Rayleigh quotient

$$\lambda_{\text{raw}} = \frac{q_{\text{pre}}^T A q_{\text{pre}}}{q_{\text{pre}}^T(-B)q_{\text{pre}}}.$$

When this estimate is valid and positive, the generated shift is $\lambda_{\text{raw}}/10^6$. Otherwise a diagonal-ratio fallback is used; if that also fails, 10^{-12} is used.

17.6.3 Typical DSL

Syntax

```
*LOADCASE, TYPE=LINEARBUCKLING, NAME=<name>
  *SUPPORTS
  <support-collector>
  *LOADS
  <preload-collector>
  *NUMEIGENVALUES
  <count>
  *SIGMA
  <shift>
```

`*LOADS` defines the preload. `*REQUESTSTIFFNESS` and `*REQUESTSTGEOM` can be used to write elastic and geometric stiffness matrices for diagnostics.

17.7 Linear Transient

`LINEARTRANSIENT` solves the linear dynamic response over time with implicit Newmark integration. The structure remains linear; contact, plasticity, and large deformation effects are not included in this loadcase.

17.7.1 Governing equation

The physical equation is

$$M\ddot{u}(t) + C\dot{u}(t) + Ku(t) = f(t).$$

With Nullspace constraints, this becomes

$$M_r\ddot{q}(t) + C_r\dot{q}(t) + K_r q(t) = r(t),$$

with

$$M_r = T^T M T, \qquad C_r = T^T C T, \qquad K_r = T^T K T.$$

Loads with amplitudes are assembled as

$$f(t) = \sum_j a_j(t) f_j.$$

For Rayleigh damping,

$$C = \alpha M + \beta K.$$

17.7.2 Typical DSL

Syntax

```
*LOADCASE, TYPE=LINEARTRANSIENT, NAME=<name>
  *SUPPORTS
  <support-collector>
  *LOADS
  <load-collector>
  *TIME
  <t_start>, <t_end>, <dt>
  *NEWMARK
  <beta>, <gamma>
  *DAMPING, TYPE=RAYLEIGH
  <alpha>, <beta>
  *WRITEEVERY, TYPE=STEPS
  <n>
```

The transient-specific keywords `*TIME`, `*NEWMARK`, `*DAMPING`, `*WRITEEVERY`, and `*INITIALVELOCITY` are described in Chapter 16.

17.8 Typical command overview

Loadcase	Typical commands
`LINEARSTATIC`	`*SUPPORTS`, `*LOADS`, `*SOLVER`, `*CONSTRAINTMETHOD`, optional `*INERTIARELIEF`, `*REBALANCELOADS`, `*REQUESTSTIFFNESS`, `*CONSTRAINTSUMMARY`.
`LINEARSTATICTOPO`	Linear-static commands plus `*TOPODENSITY`, `*TOPOORIENT`, `*TOPOEXPONENT`.
`EIGENFREQ`	`*SUPPORTS`, `*CONSTRAINTMETHOD`, `*NUMEIGENVALUES`.
`LINEARBUCKLING`	`*SUPPORTS`, `*LOADS`, `*CONSTRAINTMETHOD`, `*NUMEIGENVALUES`, optional `*SIGMA`, `*REQUESTSTIFFNESS`, `*REQUESTSTGEOM`.
`LINEARTRANSIENT`	`*SUPPORTS`, `*LOADS`, `*SOLVER`, `*CONSTRAINTMETHOD`, `*TIME`, `*NEWMARK`, `*DAMPING`, `*WRITEEVERY`, `*INITIALVELOCITY`.

18 Result Format

FEMaster writes text-based `.res` files. The file is organized into loadcases and fields. Each loadcase starts with a loadcase header followed by any number of field blocks.

18.1 Loadcase blocks

Syntax

```
LOADCASE <id> <name-or-type>
```

The loadcase ID follows execution order. The following field blocks belong to that loadcase until the next loadcase header appears.

18.2 Dense fields

Syntax

```
FIELD <name> ROWS=<rows> COLS=<cols>
<v11> <v12> ...
```

`ROWS` gives the number of rows. `COLS` gives the number of values per row. For nodal fields, a row typically corresponds to a node-position row. For element fields, a row corresponds to an element-position row.

18.3 Indexed fields

Syntax

```
FIELD <name> ROWS=<rows> INDEX_COLS=<i> VALUE_COLS=<v>
<index...> <value...>
```

Some result quantities are not simple node or element fields. Indexed fields use index columns to identify element, integration point, section point, or local result location, followed by value columns.

18.4 Common field names

Loadcase	**Typical fields**
Linear Static	`DISPLACEMENT`, `STRAIN`, `STRESS`, `EXTERNAL_FORCES`, `REACTION_FORCES`, shell and section fields.
Linear Static Topology	Static fields plus `COMPLIANCE`, `DENS_GRAD`, `VOLUME`, `DENSITY`, and optional orientation fields.
Eigenfrequency	`MODE_SHAPE_i`, `PARTICIPATION_i`, `EIGENVALUES`, `EIGENFREQUENCIES`, `FREQUENCIES`.
Linear Buckling	`BUCKLING_MODE_i`, `BUCKLING_FACTORS`.
Linear Transient	`DISPLACEMENT_k`, `VELOCITY_k`, `ACCELERATION_k` for written time steps.

A six-component nodal motion field uses `Ux, Uy, Uz, Rx, Ry, Rz`. A six-component stress or strain field uses the engineering order x, y, z, yz, zx, xy. These conventions must not be confused.

19 Examples

19.1 Linear static

See Section 17.3 for the governing equation and a minimal static loadcase.

19.2 Eigenfrequency

Example

```
*LOADCASE, TYPE=EIGENFREQ
  *SUPPORTS
  FIX
  *NUMEIGENVALUES
  12
```

19.3 Linear buckling

Example

```
*LOADCASE, TYPE=LINEARBUCKLING
  *SUPPORTS
  FIX
  *LOADS
  PRELOAD
  *NUMEIGENVALUES
  6
  *SIGMA
  0.0
  *REQUESTSTGEOM, FILE=buckling_geom
```

19.4 Transient load with amplitude

Example

```
*AMPLITUDE, NAME=RAMP, TYPE=LINEAR
0.0, 0.0
1.0, 1.0

*CLOAD, LOAD_COLLECTOR=F_RAMP, AMPLITUDE=RAMP
TIP, 0, 0, -1000, 0, 0, 0

*LOADCASE, TYPE=LINEARTRANSIENT
  *SUPPORTS
  FIX
  *LOADS
  F_RAMP
  *TIME
  0.0, 1.0, 0.01
  *NEWMARK
  0.25, 0.5
  *WRITEEVERY, TYPE=STEPS
  10
```

19.5 Point mass

Example

```
*NSET, NAME=MASS_NODE
100

*POINTMASS, NSET=MASS_NODE
5.0, 0.02, 0.02, 0.04, 0, 0, 0, 0, 0, 0
```

A Mathematical Derivations

This appendix collects derivations that are useful for understanding the topology-related quantities written by FEMaster. The main reference chapters describe how to use the DSL commands. This appendix gives the mathematical background for compliance, density sensitivities, and orientation sensitivities.

A.1 Element stiffness

For a linear elastic finite element, the unmodified element stiffness is

$$K_{e,0} = \int_{\Omega_e} B^T D B \, d\Omega.$$

B is the strain-displacement matrix, D is the constitutive matrix, and Ω_e is the physical element domain. With an isoparametric mapping from natural coordinates $r = (\xi, \eta, \zeta)$, the integral is evaluated as

$$K_{e,0} = \int_{\Omega'_e} B(r)^T D B(r) \, |\det J(r)| \, d\xi \, d\eta \, d\zeta.$$

For numerical integration with quadrature points r_q and weights w_q, this becomes

$$K_{e,0} \approx \sum_{q=1}^{Q} w_q B(r_q)^T D B(r_q) \, |\det J(r_q)|.$$

A.2 Compliance

For a linear static analysis,

$$Ku = f.$$

The external work measure commonly used for topology optimization is the compliance

$$C = f^T u.$$

Using the equilibrium equation $f = Ku$, this can also be written as

$$C = u^T K u.$$

Because the global stiffness matrix is assembled from element stiffness matrices, compliance can be decomposed into element contributions:

$$C = \sum_{e=1}^{n_e} u_e^T K_e u_e.$$

This expression is the basis for element-wise topology sensitivities. u_e is the element displacement vector extracted from the global solution.

A.3 Density penalization

In a density-based topology formulation, an element density variable ρ_e scales the stiffness contribution. With a penalization exponent p,

$$K_e(\rho_e) = \rho_e^p K_{e,0}.$$

The exponent p makes intermediate densities less efficient and therefore encourages designs closer to a solid-void distribution.

The element compliance contribution is

$$C_e(\rho_e) = u_e^T K_e(\rho_e) u_e = \rho_e^p u_e^T K_{e,0} u_e.$$

If u_e is held fixed for the local sensitivity expression, the derivative is

$$\frac{\partial C_e}{\partial \rho_e} = p\rho_e^{p-1} u_e^T K_{e,0} u_e.$$

Depending on the optimization convention, the objective derivative may be reported with the opposite sign because minimizing compliance through the equilibrium equation often uses the adjoint result

$$\frac{dC}{d\rho_e} = -u_e^T \frac{\partial K_e}{\partial \rho_e} u_e = -p\rho_e^{p-1} u_e^T K_{e,0} u_e.$$

The sign convention should therefore be interpreted together with the optimizer interface and the field name written by the analysis.

A.4 Orientation-dependent stiffness

For orientation-based topology or material steering, the constitutive matrix is rotated before the element stiffness is evaluated. Let the material orientation be represented by three angles

$$\alpha = (\alpha_1, \alpha_2, \alpha_3).$$

The elementary rotation matrices are

$$R_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha_1 & -\sin\alpha_1 \\ 0 & \sin\alpha_1 & \cos\alpha_1 \end{bmatrix},$$

$$R_2 = \begin{bmatrix} \cos\alpha_2 & 0 & \sin\alpha_2 \\ 0 & 1 & 0 \\ -\sin\alpha_2 & 0 & \cos\alpha_2 \end{bmatrix}, \qquad R_3 = \begin{bmatrix} \cos\alpha_3 & -\sin\alpha_3 & 0 \\ \sin\alpha_3 & \cos\alpha_3 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

For the convention used here, the combined rotation is

$$R = R_1 R_2 R_3.$$

The multiplication order must be used consistently. Different conventions can describe the same physical orientation with different angle triples.

A.5 Voigt transformation

The strain vector uses engineering shear strains,

$$\varepsilon = \begin{bmatrix} \varepsilon_{xx} & \varepsilon_{yy} & \varepsilon_{zz} & 2\varepsilon_{yz} & 2\varepsilon_{xz} & 2\varepsilon_{xy} \end{bmatrix}^T,$$

and the stress vector is

$$\sigma = \begin{bmatrix} \sigma_{xx} & \sigma_{yy} & \sigma_{zz} & \sigma_{yz} & \sigma_{xz} & \sigma_{xy} \end{bmatrix}^T.$$

The strain transformation is written as

$$\varepsilon_m = T(R)\varepsilon_g,$$

where T is the 6×6 Voigt transformation matrix built from the entries of R. The rotated constitutive matrix used in global strain coordinates is

$$D_{\text{rot}} = T^T D T.$$

The orientation-dependent element stiffness is therefore

$$K_e(\rho_e, \alpha_e) = \rho_e^p \int_{\Omega_e} B^T D_{\text{rot}}(\alpha_e) B \, d\Omega.$$

With quadrature,

$$K_e(\rho_e, \alpha_e) \approx \rho_e^p \sum_{q=1}^{Q} w_q B(r_q)^T T(\alpha_e)^T D T(\alpha_e) B(r_q) |\det J(r_q)|.$$

A.6 Orientation sensitivity

For one element, the compliance contribution is

$$C_e = \rho_e^p \sum_{q=1}^{Q} w_q \varepsilon_q^T T^T D T \varepsilon_q |\det J_q|, \qquad \varepsilon_q = B(r_q) u_e.$$

Only T depends on the orientation angles, so the derivative with respect to angle α_j is

$$\frac{\partial C_e}{\partial \alpha_j} = \rho_e^p \sum_{q=1}^{Q} w_q \varepsilon_q^T \left(\frac{\partial T^T}{\partial \alpha_j} D T + T^T D \frac{\partial T}{\partial \alpha_j} \right) \varepsilon_q |\det J_q|.$$

If D is symmetric, the scalar expression can also be evaluated as

$$\frac{\partial C_e}{\partial \alpha_j} = 2\rho_e^p \sum_{q=1}^{Q} w_q \varepsilon_q^T T^T D \frac{\partial T}{\partial \alpha_j} \varepsilon_q |\det J_q|,$$

provided the same transformation convention is used.

A.7 Rotation derivatives

The derivatives of the elementary rotation matrices are

$$\frac{\partial R_1}{\partial \alpha_1} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & -\sin\alpha_1 & -\cos\alpha_1 \\ 0 & \cos\alpha_1 & -\sin\alpha_1 \end{bmatrix},$$

$$\frac{\partial R_2}{\partial \alpha_2} = \begin{bmatrix} -\sin\alpha_2 & 0 & \cos\alpha_2 \\ 0 & 0 & 0 \\ -\cos\alpha_2 & 0 & -\sin\alpha_2 \end{bmatrix},$$

$$\frac{\partial R_3}{\partial \alpha_3} = \begin{bmatrix} -\sin\alpha_3 & -\cos\alpha_3 & 0 \\ \cos\alpha_3 & -\sin\alpha_3 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

For $R = R_1 R_2 R_3$, the total rotation derivatives are

$$\frac{\partial R}{\partial \alpha_1} = \frac{\partial R_1}{\partial \alpha_1} R_2 R_3,$$

$$\frac{\partial R}{\partial \alpha_2} = R_1 \frac{\partial R_2}{\partial \alpha_2} R_3,$$
$$\frac{\partial R}{\partial \alpha_3} = R_1 R_2 \frac{\partial R_3}{\partial \alpha_3}.$$

The derivative $\partial T/\partial \alpha_j$ is obtained by differentiating every entry of $T(R)$ with the product rule. For example, an entry $R_{ab}R_{cd}$ contributes

$$\frac{\partial}{\partial \alpha_j}\left(R_{ab}R_{cd}\right) = \frac{\partial R_{ab}}{\partial \alpha_j} R_{cd} + R_{ab} \frac{\partial R_{cd}}{\partial \alpha_j}.$$

This product-rule structure is the key point of the orientation derivative: once R, $\partial R/\partial \alpha_j$, T, and $\partial T/\partial \alpha_j$ are known, the sensitivity follows directly from the quadrature expression above.